



PRÁTICAS DE SISTEMAS DIGITAIS - T02

Relatório de Práticas em Sistemas Digitais Primeira Unidade

João Carlos Cruz Santos, João Victor Carvalho Simões, Otávio
Augusto Batista Santos Filho

¹Departamento de Computação – Universidade Federal de Sergipe (UFS) São
Cristóvão, SE – Brasil

{joaoccs, vonvictor, 177013ot}@academico.ufs.br

Relatório da prática 08 – Lab 1

São Cristóvão, Sergipe 2025

SUMÁRIO

- 0. Resumo**
- 1. Introdução**
- 2. Entendendo o Circuito**
 - 2.1 Entradas e Portas Lógicas**
 - 2.2 Expressões Lógicas das Saídas**
 - 2.3 Tabela-Verdade do Somador Completo**
- 3. Prototipação na Plataforma Pitanga**
 - 3.1 Modelagem em Verilog Estrutural**
 - 3.2 Modelagem em Verilog Dataflow**
 - 3.3 Testagem e Validação**
- 4. Projeto Hierárquico – Somador de 4 Bits**
- 5. Hora Trabalho – Relatório Individual**
- 6. Referências Bibliográficas**

0. Resumo

Este relatório apresenta o desenvolvimento, a implementação e a análise do circuito **Somador Completo**, utilizando a linguagem de descrição de hardware **Verilog** nos níveis **estrutural** e **dataflow**, bem como a aplicação dos conceitos de **projeto hierárquico** por meio da construção de um **somador de 4 bits**. As atividades foram realizadas no simulador **Pitanga**, da InPlace, permitindo a síntese, mapeamento e visualização prática do funcionamento dos circuitos digitais. O trabalho proporcionou o aprofundamento do entendimento dos princípios fundamentais dos sistemas digitais, além de consolidar conceitos relacionados à modelagem em HDL, simulação, síntese e verificação automática de projetos lógicos.

1. Introdução

O presente relatório tem como objetivo descrever e analisar as atividades laboratoriais referentes ao estudo do **Somador Completo**, enfatizando a utilização da linguagem **Verilog HDL** em nível **dataflow** e **estrutural**, bem como a aplicação do conceito de **projeto hierárquico**.

Por meio deste experimento, buscou-se compreender de forma prática os fundamentos dos **circuitos combinacionais**, a síntese lógica e o mapeamento em FPGA, utilizando a plataforma virtual **Pitanga**, da InPlace.

O desenvolvimento do somador completo e sua expansão para um somador de

4 bits possibilitou consolidar conceitos essenciais dos sistemas digitais, como propagação de carry, modularização de circuitos, reutilização de módulos e validação automática por meio de simulação.

2. Entendendo o Circuito

2.1 Entradas e Portas Lógicas Utilizadas

O circuito **Somador Completo** possui três entradas:

- **A**
- **B**
- **Cin (carry-in)**

E duas saídas:

- **S (soma)**
- **Cout (carry-out)**

As portas lógicas utilizadas são:

- Duas portas **AND**
- Uma porta **OR**
- Duas portas **XOR**

Essas portas combinam-se para implementar corretamente a soma binária de três bits.

2.2 Expressões Lógicas das Saídas

As expressões lógicas equivalentes das saídas são:

O comportamento das saídas pode ser interpretado da seguinte forma:

- **S = 1** quando há um número ímpar de entradas em nível alto.
- **Cout = 1** quando duas ou mais entradas estão em nível alto.

Assim:

- Uma entrada em nível alto $\rightarrow$ apenas **S = 1**
- Duas entradas em nível alto $\rightarrow$ apenas **Cout = 1**
- Três entradas em nível alto $\rightarrow$ **S = 1** e **Cout = 1**

2.3 Tabela-Verdade do Somador Completo

A	B	Cin	Cout	S
1	1	1	1	1
1	1	0	1	0
1	0	1	1	0
1	0	0	0	1
0	1	1	1	0
0	1	0	0	1
0	0	1	0	1
0	0	0	0	0

3. Prototipação na Plataforma Pitanga

3.1 Modelagem em Verilog Estrutural

Inicialmente, o circuito foi descrito utilizando **Verilog estrutural**, caracterizado pelo uso explícito das portas lógicas. Esse tipo de descrição apresenta baixo nível de abstração, sendo ideal para compreender o funcionamento interno do circuito.

O mapeamento dos sinais foi realizado conforme:

- **A** $\rightarrow$ **SW0**
- **B** $\rightarrow$ **SW1**
- **Cin** $\rightarrow$ **SW2**
- **S** $\rightarrow$ **LED0**
- **Cout** $\rightarrow$ **LED1**

3.2 Modelagem em Verilog Dataflow

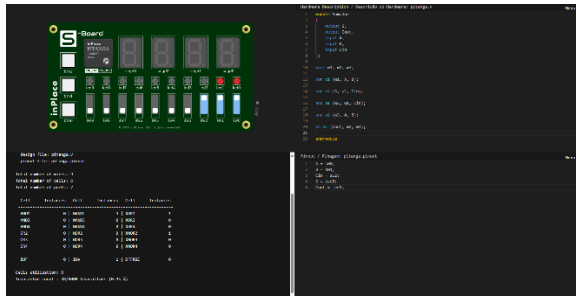
Em seguida, o circuito foi descrito no nível **dataflow**, que apresenta maior grau de abstração. Nesse modelo, as saídas são descritas diretamente a partir das expressões booleanas, sem a necessidade de instanciar explicitamente as portas lógicas.

```
Hardware Description / Descrição do Hardware: pitanga.v
1  module somador_dataflow
2  (
3      output S,
4      output Cout,
5      input A,
6      input B,
7      input Cin
8  );
9
10 assign Cout = ((A ^ B) & Cin) | (A & B);
11 assign S = (A ^ B) ^ Cin;
12
13 endmodule
```

Além disso, utilizou-se a técnica de **instanciamento**, permitindo a reutilização de módulos previamente definidos, facilitando a implementação de circuitos mais complexos e promovendo organização, modularidade e reaproveitamento de código.

3.3 Testagem e Validação

A testagem foi realizada por meio da simulação no ambiente **Pitanga**, utilizando o mapeamento descrito anteriormente. Os resultados observados foram plenamente compatíveis com a tabela-verdade do somador completo.



- Para apenas uma entrada em nível alto $\rightarrow S = 1, Cout = 0$
- Para duas entradas em nível alto $\rightarrow S = 0, Cout = 1$
- Para três entradas em nível alto $\rightarrow S = 1, Cout = 1$

```
Pinout / Pinagem: pitanga.pinout
1 A = sw0;
2 B = sw1;
3 Cin = sw2;
4 S = led0;
5 Cout = led1;
```

Esses resultados confirmaram a correta implementação tanto no modelo estrutural quanto no dataflow.

4. Projeto Hierárquico – Somador de 4 Bits

Após a implementação do somador completo, foi desenvolvido um **somador de 4 bits**, utilizando a técnica de **projeto hierárquico**, por meio do encadeamento de quatro módulos de somador completo.

Esse circuito realiza a soma de dois números binários de 4 bits, considerando a propagação do carry entre os estágios. A validação foi realizada por meio de testes práticos, como:

- $10 + 7 \rightarrow 1010 + 0111 = 10001$
- $9 + 6 \rightarrow 1001 + 0110 = 1111$

- $15 + 14 \rightarrow 1111 + 1110 = 11101$

Todos os testes apresentaram resultados corretos, comprovando o funcionamento adequado do circuito.

Pinout / Pinagem: pitanga.pinout

```
1 a[0] = sw0;
2 a[1] = sw1;
3 a[2] = sw2;
4 a[3] = sw3;
5
6 b[0] = sw4;
7 b[1] = sw5;
8 b[2] = sw6;
9 b[3] = sw7;
10
11 s[0] = led0;
12 s[1] = led1;
13 s[2] = led2;
14 s[3] = led3;
15 s[4] = led4;
16
17 module somador_dataflow
18 (
19     output S,
20     output Cout,
21     input A,
22     input B,
23     input Cin
24 );
25
26 assign Cout = ((A ^ B) & Cin) | (A & B);
27 assign S = (A ^ B) ^ Cin;
28
29 endmodule
30
31 module somador_4bits
32 (
33     output [4:0]s,
34     input [3:0]a,
35     input [3:0]b
36 );
37
38 wire w1, w2, w3;
39
40 somador_dataflow fa0
41 (
42     .S(s[0]),
43     .Cout(w1),
44     .A(a[0]),
45     .B(b[0]),
46     .Cin(1'b0)
47 );
48
49 somador_dataflow fa1
50 (
51     .S(s[1]),
52     .Cout(w2),
53     .A(a[1]),
54     .B(b[1]),
55     .Cin(w1)
56 );
57
58 somador_dataflow fa2
59 (
60     .S(s[2]),
61     .Cout(w3),
62     .A(a[2]),
63     .B(b[2]),
64     .Cin(w2)
65 );
66
67 somador_dataflow fa3
68 (
69     .S(s[3]),
70     .Cout(s[4]),
71     .A(a[3]),
72     .B(b[3]),
73     .Cin(w3)
74 );
75
76 endmodule
```

5. Hora Trabalho – Relatório Individual

a) Ferramenta Icarus Verilog

O **Icarus Verilog** é um compilador e simulador de código aberto para a linguagem Verilog. Ele permite compilar, simular e validar projetos de hardware antes da implementação física em FPGA ou ASIC. Seu principal objetivo é verificar o comportamento funcional do circuito por meio de simulação, evitando erros de projeto e reduzindo custos de desenvolvimento.

b) Ferramenta GTKWave

O **GTKWave** é um visualizador de formas de onda utilizado para analisar os sinais gerados durante a simulação. Ele permite observar transições de sinais no domínio do tempo, facilitando a depuração e validação do funcionamento correto do circuito.

c) Características do Verilog para Simulação

A linguagem Verilog suporta:

- Múltiplos níveis de abstração (comportamental, RTL e estrutural);
- Execução concorrente de blocos;
- Sensibilidade a eventos;
- Tipos de dados específicos (wire, reg);
- Lógica de quatro estados (0, 1, X, Z);
- Recursos para testbench, como `initial`, `#delay`, `$display`, `$dumpfile`, `$finish`.

d) Estratégia de Simulação Automática

A estratégia consiste na criação de um **testbench**, responsável por aplicar automaticamente todas as combinações possíveis de entrada, monitorar as saídas e verificar sua correção. Essa abordagem permite identificar erros rapidamente, realizar testes exaustivos e validar completamente o circuito sem depender de observação manual.

6. Referências Bibliográficas

- [1] TOCCI, R. J.; WIDMER, N. S.; MOSS, G. L. **Sistemas Digitais: Princípios e Aplicações**. 12. ed. São Paulo: Pearson Education do Brasil, 2018.
- [2] UYEMURA, J. P. **Sistemas Digitais: Uma Abordagem Integrada**. São Paulo: Thomson/Pioneira, 2002.
- [3] QIU, R. et al. **AutoBench: Automatic Testbench Generation and Evaluation Using LLMs for HDL Design**. Proceedings of MLCAD '24, ACM/IEEE, 2024.
- [4] ZHANG, Q. et al. **TestBench: Evaluating Class-Level Test Case Generation Capability of Large Language Models**. arXiv:2409.17561, 2024.
- [5] SCHULZ, R. **Como Simular – Introdução à Verilog**. UNICAMP. Disponível em: <https://www.ic.unicamp.br/~rodolfo/Cursos/verilog/ComoSimular/>.